

character), followed by the name of the member, as in `MyModule.abs(-42)`. To use the `toString` member on the object 42, we'd use `42.toString`. The implementations of members within an object can refer to each other unqualified (without prefixing the object name), but if needed they have access to their enclosing object using a special name: `this`.²

Note that even an expression like `2 + 1` is just calling a member of an object. In that case, what we're calling is the `+` member of the object 2. It's really syntactic sugar for the expression `2.+(1)`, which passes 1 as an argument to the method `+` on the object 2. Scala has no special notion of *operators*. It's simply the case that `+` is a valid method name in Scala. Any method name can be used infix like that (omitting the dot and parentheses) when calling it with a single argument. For example, instead of `MyModule.abs(42)` we can say `MyModule abs 42` and get the same result. You can use whichever you find more pleasing in any given case.

We can bring an object's member into scope by *importing* it, which allows us to call it unqualified from then on:

```
scala> import MyModule.abs
import MyModule.abs

scala> abs(-42)
res0: 42
```

We can bring *all* of an object's (nonprivate) members into scope by using the underscore syntax:

```
import MyModule._
```

2.4 Higher-order functions: passing functions to functions

Now that we've covered the basics of Scala's syntax, we'll move on to covering some of the basics of writing functional programs. The first new idea is this: *functions are values*. And just like values of other types—such as integers, strings, and lists—functions can be assigned to variables, stored in data structures, and passed as arguments to functions.

When writing purely functional programs, we'll often find it useful to write a function that accepts other functions as arguments. This is called a *higher-order function* (HOF), and we'll look next at some simple examples to illustrate. In later chapters, we'll see how useful this capability really is, and how it permeates the functional programming style. But to start, suppose we wanted to adapt our program to print out both the absolute value of a number *and* the factorial of another number. Here's a sample run of such a program:

```
The absolute value of -42 is 42
The factorial of 7 is 5040
```

² Note that in this book, we'll use the term *function* to refer more generally to either so-called standalone functions like `sqrt` or `abs`, or members of some class, including methods. When it's clear from the context, we'll also use the terms *method* and *function* interchangeably, since what matters is not the syntax of invocation (`obj.method(12)` vs. `method(obj, 12)`), but the fact that we're talking about some parameterized block of code.